

Modern Cloud Computing

Lecture 5: Cloud Hardware and Software

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- WSC Hardware
- Virtualization
- Cloud Software

WSC Hardware

Challenges: The Evolution of Computing

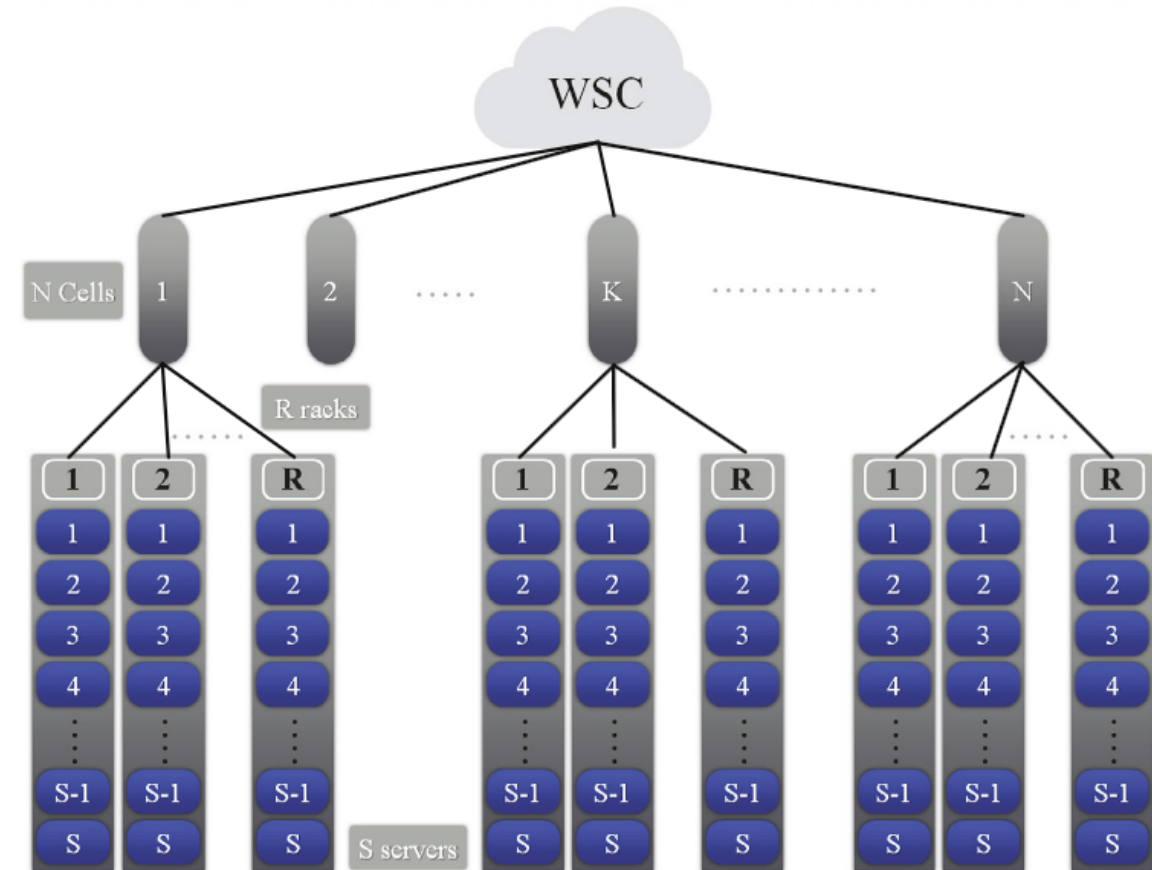
Warehouse-Scale Computers (WSCs) serve millions of users with hundreds of thousands of processors, built from **inexpensive, off-the-shelf** components

Challenges:

- Complex software stacks; off-the-shelf parts have low **mean-time-to-failure**
- **99.99% availability** $\Rightarrow$ < one hour of downtime per year
- Needs hardware redundancy **plus** software error detection and recovery

WSC Organization

- WSCs form the backbone of Google, Amazon, and other CSPs
- Hierarchically organized: **50,000–100,000 processors**
- Exploit **request-level** and **data-level** parallelism
- At the heart is a **hierarchy of networks** connecting servers, racks, and cells/arrays



The Network Hierarchy

- **Rack:** ~48 servers connected by a 48-port, 10 Gbps Ethernet (GE) switch
 - 2–8 uplink ports connect the rack to a cell
 - **Oversubscription ratio** = internal/external ports $\approx$ 6 to 24
 - Same-rack processes get much higher bandwidth and lower latency
- **Cell (array):** many racks joined by an expensive **array switch**
 - Switch cost scales as $O(n^2)$ with n ports
 - An array switch can support up to **30 racks**

WSCs Are Expensive

- The cost-**performance** is what makes WSCs appealing
- A WSC costs on the order of **\$150 million**
- Case-study capital expenditure: **\$167,510,000**
 - \$66,700,000 for **45,978 servers**
 - \$12,810,000 for the interconnect (1,150 rack switches, 22 cell switches, ...)
- Operating (energy) cost is significant — the facility is expected to use **8 MW**

Brawny vs. Wimpy Cores

Brawny cores

- High single-core performance
- High power dissipation

Wimpy cores

- Less powerful, but consume less power
- Need to spawn **more threads** per task

The Trouble with Wimpy Cores

- More threads → **explicit parallelization** raises development cost
- More threads → higher **response time** (barrier synchronization: all wait for the slowest)
- **DRAM cost rises** as kernel/system processes consume more aggregate memory
- Data structures may not fit a single wimpy-core machine's memory
- Larger thread counts → higher **scheduling overhead**

Measuring WSC Performance

- Even slight improvements → **large cost savings**
- No "killer" application; workloads are **very diverse**; can't experiment/simulate at scale
- Only alternative: **profile realistic workloads** with low-overhead monitoring
- **Google-Wide-Profiling (GWP)**: low-overhead tool using **random sampling**
 - Selects servers, uses **Perf**, collects **callstacks**, aggregates into a database

The "WSC Architecture Tax"

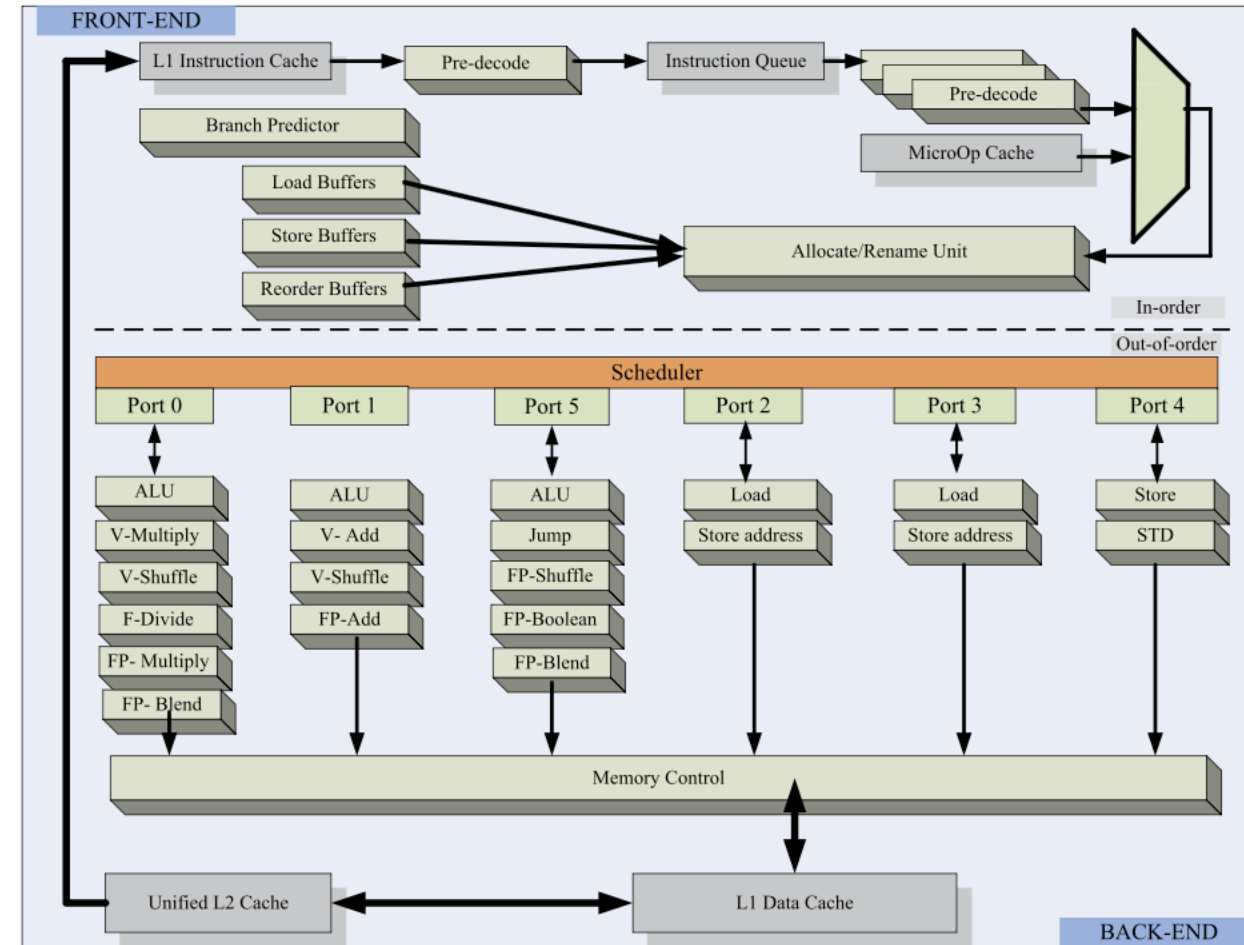
- Common procedures dominate CPU cycles across diverse workloads
- **Communication** overhead (RPCs, serialization, deserialization, compression):
 - Consumes **22–27%** of CPU cycles
 - $\sim\frac{1}{3}$ of RPCs used by cluster management (load balancing, encryption)
- `memmove` / `memcpy` account for **4–5%** of the tax
- Compression, hashing, memory allocation: $>\frac{1}{4}$ of the tax
- $\sim\frac{1}{5}$ of cycles spent in the scheduler and kernel

Memory Footprint & Locality

- Binaries of **hundreds of MB** are common; little spatial or temporal locality
- Memory footprint grows **~30% per year**; instruction-cache footprint **~2% per year**
- **Memory latency** limits Instruction-Level Parallelism (ILP)
- Stall cycles from **cache misses** dominate; data-cache misses are the biggest contributor
- Linked data structures cause indirect addressing that **defeats prefetching**

Modern Core Microarchitecture

- **Front-end:** processes instructions in **order** (fetch, decode, branch prediction)
- **Back-end:** the instruction scheduler dispatches to execution units **out of order**
- Five scheduler ports feed **ALUs** and **load/store** units; vector (V) and floating-point (FP) ops go to ALUs



Top-Down Performance Analysis

- **Top-Down methodology** (Intel): a hierarchical organization of event-based metrics identifying dominant bottlenecks
- The **micro-op (μ op) queue** separates front-end from back-end
- μ op pipeline slots classified as:
 - **Retiring** (the only useful work), **Front-end bound**, **Bad speculation**, **Back-end bound**
- WSC workloads: **higher front-end boundedness** and **lower retirement rates** than SPEC CPU 2006

Key Performance Findings

- **Back-end bound** dominates → limits active cores per cycle
 - On a 6-core Ivy Bridge, only 1–2 cores active in 72% of cycles
- Low DRAM bandwidth utilization (avg 31%, max 68%)
- Median CPU bandwidth utilization as low as **10%**
- Large working sets → high instruction-cache **MPKI** (misses per kilo instructions)

Virtualization

Processor Virtualization & VMs

- Multiple OS instances — pioneered by **IBM in the early 1970s**, revived for the cloud
- A **guest OS** runs above a **hypervisor**
- Multiple VMs → better resource sharing, higher utilization (complementary demands, less idle time)

VM migration is straightforward:

1. Stop the VM → 2. Save state as a file → 3. Transport the file → 4. Restart the VM

Benefits developers (familiar environment, OS of choice) and CSPs (low-cost delivery, higher profit).

The Cost of Virtualization

- Increases **complexity**; undesirable side-effects on **performance and security**
- Processor sharing controlled by a new layer: the **hypervisor** (Virtual Machine Monitor)

What Is a Hypervisor?

- Software that **securely partitions** resources into one or more VMs
- A **guest OS** runs above the hypervisor (user mode); the hypervisor runs in **kernel mode**
- Several operating systems run **concurrently**; events in one VM don't affect others

A hypervisor enables:

- **Multiple services** sharing one platform
- **Live migration** across platforms
- **System modification** with backward compatibility

Mechanism: a guest OS's **privileged instruction** is **trapped** to enforce correctness, safety, and isolation.

Containers

- Based on **operating-system-level virtualization**, not hardware virtualization
- Portable; resource use can be **limited**; more transparent and easier to manage than VMs

Today: **Docker** (ease of use) and **Google Kubernetes** (performance-oriented) dominate.

Benefits of Containers

- Support **portability** — run independently of the environment
- Enable **application-centric** management
- Optimal deployment philosophy — apps split into smaller, independent pieces
- Support **higher resource utilization**
- Lead to **predictable** application performance

Cloud Software

SPMD & Coarse-Grained Parallelism

- **SPMD** = Single-Program-Multiple-Data: split a large dataset into segments processed independently and concurrently by the same program
- Example: convert 10^9 images via 1,000 segments of 10^6 images on 1,000 processors → time cut by ~3 orders of magnitude
- Ideal for the cloud: keeps systems busy, raises utilization
- **MapReduce** (2004) extends SPMD: a **Map** phase then a **Reduce** phase merging partial results

The MapReduce Programming Model

Inspired by `map` and `reduce` in Lisp and other functional languages.
The user writes **two functions**; the library does everything else.

```
map      (k1, v1)      → list(k2, v2)
reduce (k2, list(v2))  → list(v2)
```

- **map** takes one input pair and emits intermediate key/value pairs
- the library **groups all values sharing an intermediate key**
- **reduce** merges the values for one key into a (usually smaller) result
- values arrive through an **iterator**, so a key's value list need not fit in memory

Canonical Example: Word Count

```
map(String key, String value):           // key: document name
    for each word w in value:           // value: document contents
        EmitIntermediate(w, "1");

reduce(String key, Iterator values):      // key: a word
    int result = 0;                      // values: a list of counts
    for each v in values:
        result += ParseInt(v);
    Emit(AsString(result));
```

- `map` emits each word with a count of 1; `reduce` sums those counts
- The user also fills in a **specification object** (input/output files, tuning parameters) and links against the MapReduce library
- No code for parallelism, retries, or data movement appears anywhere

Execution Overview: Step by Step

1. Input is split into **M** pieces of 16–64 MB, and many copies of the program are started.
2. One copy becomes the **master**, while the rest are **workers**. The master assigns **M** map tasks and **R** reduce tasks.
3. Each map worker reads its input split, calls `map`, and buffers the generated key–value pairs in memory.
4. The buffered data is periodically written to **local disk**, partitioned into **R** regions using: `hash(key) mod R`
5. Each reduce worker fetches its assigned partition from every map worker's local disk (via RPC) and **sorts the data by key**.
6. The reduce worker iterates through the sorted keys, calls `reduce` once for each unique key, and appends the results to its output file.
7. After all tasks finish, the master notifies the user program. The final output consists of **R output files**, which are often used directly as the input to the next MapReduce job.

Hadoop

A framework that allows for the distributed processing of large data sets across clusters of computers

The four modules

- HDFS: distributed file system, the open counterpart of GFS
- YARN: cluster resource management and scheduling
- MapReduce: the batch execution engine, now just one YARN application
- Common: shared utilities, RPC, configuration, security

HDFS

Files are partitioned into large **blocks** (128 MB by default), with each block replicated across **DataNodes** (3 replicas by default).

Roles

- **NameNode** — Maintains the filesystem namespace and block metadata in memory; does not store user data.
- **DataNodes** — Store block replicas, serve client read/write requests, and periodically send **heartbeats** and **block reports**.

Properties

- Clients obtain block locations from the **NameNode** but transfer data directly with **DataNodes**, separating metadata management from data access.
- **Rack-aware replication** places replicas across racks to improve fault tolerance and availability.
- Missing heartbeats trigger block re-replication; **HDFS High Availability (HA)** employs a standby **NameNode** for failover.

YARN and Job Execution

Components

- **ResourceManager** — Global resource manager that allocates **containers** (CPU and memory) across the cluster.
- **NodeManager** — Per-node service responsible for launching, monitoring, and managing containers.
- **ApplicationMaster** — Application-specific coordinator that negotiates containers and manages task execution for a single job.

Key Advantages

- Decouples resource management from computation
- Supports pluggable scheduling policies
- Isolates application failures, preventing a failed **ApplicationMaster** from affecting the entire cluster.

Narrow vs. Wide Dependencies

Narrow dependencies

- One parent partition per child
- Allow **pipelined** execution
- Recovery: recompute only lost parent

Wide dependencies

- Multiple children per parent
- Data must be **shuffled** across nodes
- Recovery is **more complex**

Docker Containers

- Made containers **easy to use** — standard APIs, portable across all environments

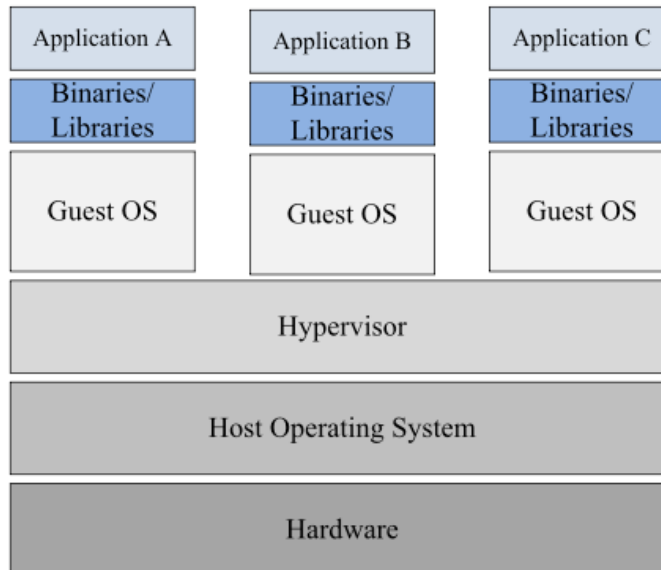
"Docker containers wrap a piece of software in a complete filesystem that contains everything needed to run... guaranteeing the software runs the same, regardless of environment."

- **Lightweight**, cost-effective; multiple containers **share the OS kernel** → smaller footprint, shorter start-up than VMs
- Increased **productivity**: choose the best language/tools; efficient up/down scaling

VMs vs. Docker Containers

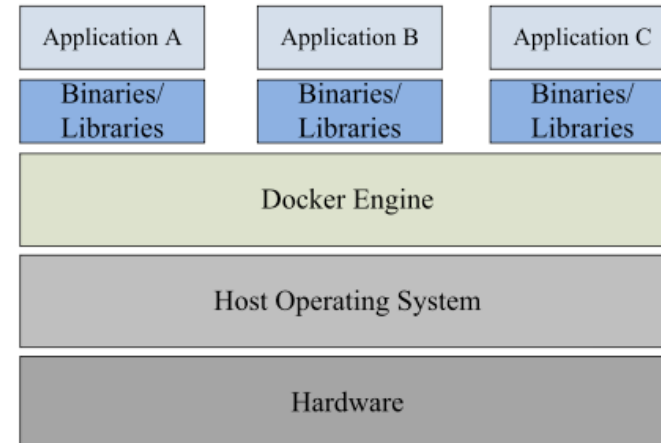
Virtual Machines

- Each app: Binaries/Libraries + **Guest OS**
- **Hypervisor** over host OS
- Larger footprint, slower start-up



Docker Containers

- Each app: Binaries/Libraries only
- **Docker Engine** over host OS
- Share the OS kernel → light & fast



The Docker Ecosystem & CaaS

- **Image** (blueprint) → **Container** (running app); **Daemon** manages, **Client** talks to it, **Hub** is the registry
- **Base images** (Ubuntu, BusyBox) vs. **child images**; a **Dockerfile** automates image creation
- **Containers as a Service (CaaS)**: Heroku, OpenShift, CloudFoundry; **Amazon ECS**; **Google GKE** (Kubernetes-based — 2 billion containers/week at Google); **Azure** supports Docker & Mesos

Kubernetes

- Bridges clustered infrastructure and application assumptions; a **cluster manager for containers**
- Written in **Go**; lightweight, modular, portable, **extensible**
 - No built-in message buses, databases, or config language
 - Provides deployment, scaling, load balancing, logging, monitoring

Kubernetes Organization

- **Master server** manages the workload and cluster communication
- **Minions**: servers that do the actual work
- **etcd**: a lightweight distributed key-value store for configuration data
- Minions run:
 - A **docker** service to run encapsulated application containers
 - A **kubelet** service — communicates with master and etcd, updates state
 - A **proxy** service for primitive load balancing

Pods

- **Pods**: groups of containers on the **same host**; units of scheduling, deployment, scaling, replication
- Share **fate** and resources (e.g., storage volumes)
- Share a **network namespace** (IP + port space)

Replication Controllers, Labels & Services

- **Replication controllers:** handle the container lifecycle
 - Automatically **replace** pods that fail, get deleted, or are terminated
 - Supervise multiple pods across multiple nodes
- **Labels:** means to find and query containers
- **Services:** identify a set of containers performing a common function

Summary

- **WSCs** deliver cheap cycles from commodity parts; the challenge is a **mix of workloads**
- **Virtualization** (VMs) multiplexes hardware; **containers** aggregate and are lighter & faster
- WSC performance is **back-end / memory-latency bound**; the "architecture tax" is real
- **Docker** and **Kubernetes** made containers ubiquitous in the cloud